Note de cadrage — Solveur de sudoku par vision par ordinateur

1. Le groupe

Membre	Rôle
Vincent	**Modèles et données** : constitution des datasets, entraînement du détecteur de grille et du classifieur de cases (YOLO), suivi des expériences dans MLflow
Titien	**Pipeline et API** : redressement OpenCV, solveur, API FastAPI, évaluation de bout en bout, tests
Aymeric	**Application et qualité** : interface web, parcours utilisateur, tests sur photos réelles, documentation

2. Le sujet choisi

Sujet 3 : résoudre un sudoku à partir d'une photo d'une grille papier. Pourquoi : c'est le sujet qui correspond le plus à un **cas concret** pour nous : un usage réel et quotidien (une grille de journal ou de livre, prise en photo au téléphone), avec un utilisateur final clairement identifié. Le sujet combine apprentissage profond (détection, classification) et vision classique (géométrie, redressement). Le résultat est vérifiable sans ambiguïté : le sudoku est résolu ou ne l'est pas. Des données publiques existent (environ 230 photos annotées), ce qui permet de mesurer la solution sur des photos jamais vues.

3. La solution finale

Type d'application : application **web**, utilisable sur ordinateur et sur téléphone (appareil photo du navigateur). Interface en **Streamlit**, qui appelle une **API REST FastAPI** ; les modèles sont versionnés dans le **Model Registry MLflow**.

Parcours utilisateur :

1. L'utilisateur ouvre l'application et prend en photo une grille de journal ou de livre (ou importe une image).
2. Il voit la grille repérée sur sa photo (cadre et quadrillage superposés).
3. Il voit la grille lue, sous forme de tableau. Les cases dont la lecture est incertaine sont signalées et il peut corriger un chiffre mal lu.
4. Il voit la solution : les chiffres manquants sont écrits directement sur sa photo, dans les cases vides. Si la grille est illisible ou insoluble, un message l'explique au lieu de donner une solution fausse.

Fonctionnement :

- **Entrée :** une photo (JPEG ou PNG).
- **Modèle 1, détecteur YOLO :** il reçoit la photo et renvoie le cadre de la ou des grilles, avec une confiance.
- **OpenCV :** il trouve les 4 coins de la grille, la redresse en vue de dessus et la découpe en 81 cases de 64×64 pixels.
- **Modèle 2, classifieur YOLO :** il reçoit les 81 cases et renvoie, pour chacune, une probabilité pour chaque classe : vide ou 1 à 9.
- **Solveur :** il vérifie que la grille respecte les règles, corrige si besoin les cases peu sûres, puis la résout.
- **Communication application ↔ API :** l'application envoie la photo à l'API en HTTP (`POST /solve`, multipart). L'API renvoie un JSON qui contient :
  - la grille lue et la confiance de chaque case ;
  - la solution ;
  - la position des cases sur la photo (pour le dessin de la solution).

  Après une correction manuelle, l'application rappelle l'API (`POST /solve-grid`) avec la grille seulement, sans refaire la lecture de la photo.

Objectif mesuré : au moins 90 % des sudokus résolus correctement sur des photos jamais vues. Le prototype actuel atteint **97,6 % (40 photos sur 41)**, sans aucune solution fausse.